

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



TRABAJO DE INVESTIGACIÓN
DETECCIÓN DE ANOMALÍAS EN LOGS DE UN SISTEMA DE
REPORTES DE VENTAS MEDIANTE ALGORITMOS DE
MINERÍA DE DATOS

Trabajo de Investigación para la Materia de Minería de Datos

POR: ROLANDO TROCHE VENEGAS

LA PAZ - BOLIVIA

2026

ÍNDICE

1	INTRODUCCIÓN	1
2	DIAGNÓSTICO Y PLANTEAMIENTO DEL PROBLEMA	2
2.1	PLANTEAMIENTO DEL PROBLEMA	2
3	OBJETIVO DE LA INVESTIGACIÓN	4
3.1	OBJETIVO GENERAL	4
3.2	OBJETIVOS ESPECÍFICOS	4
4	ANÁLISIS DE LA INFORMACIÓN	5
4.1	FUENTE DE DATOS	5
4.2	PREPARACIÓN DE LOS DATOS	7
4.2.1	Exploración y Análisis Descriptivo	7
4.2.2	Preprocesamiento y Selección de Características	11
4.3	PROCESO DE MODELADO	12
4.3.1	Isolation Forest	12
4.3.2	Local Outlier Factor	13
4.3.3	One-Class SVM	14
4.4	EVALUACIÓN	14
5	RESULTADOS	15
5.1	COMPARACIÓN DE MODELOS	15
5.2	CURVAS ROC	16
5.3	MATRICES DE CONFUSIÓN	17
5.4	IMPORTANCIA DE VARIABLES	18
5.5	DETECCIÓN POR SERVICIO	19
6	CONCLUSIONES	20

1 INTRODUCCIÓN

En los sistemas de software modernos, los registros de eventos o *logs* constituyen una fuente fundamental de información operativa. Cada solicitud que un usuario realiza queda trazada a través de múltiples capas de servicios, generando miles de entradas por minuto que describen el comportamiento del sistema en tiempo real. La capacidad de analizar estos registros de manera eficiente determina en gran medida la velocidad con la que un equipo de operaciones puede detectar, diagnosticar y resolver incidentes (He et al., 2016).

La detección de anomalías en logs es el proceso de identificar patrones de comportamiento que se desvían significativamente de lo esperado, ya sea por degradación del rendimiento, errores en cascada, uso excesivo de recursos o exportaciones de datos inusualmente grandes (Chandola et al., 2009). En entornos distribuidos, donde una sola transacción de usuario atraviesa varios microservicios, estas anomalías pueden propagarse silenciosamente hasta convertirse en fallos críticos si no se detectan a tiempo (Xu et al., 2009).

El presente trabajo aplica técnicas de minería de datos no supervisadas —*Isolation Forest*, *Local Outlier Factor* y *One-Class SVM*— sobre registros de un sistema de reportes de ventas compuesto por tres capas de servicio: *Report Service*, *Sales API* y base de datos. El conjunto de datos contiene 30 000 registros de logs correspondientes al mes de enero de 2026, de los cuales el 4,55 % corresponde a anomalías de cuatro tipos distintos: consultas lentas a la base de datos, exportaciones de reportes de gran tamaño, alta tasa de errores y uso excesivo de recursos computacionales.

El trabajo se organiza de la siguiente manera: el Capítulo 2 diagnostica la problemática del monitoreo manual de logs; el Capítulo 3 establece los objetivos de la investigación; el Capítulo 4 describe el proceso de análisis de datos, incluyendo la fuente, preparación y modelado; el Capítulo 5 presenta los resultados comparativos de los tres modelos; y el Capítulo 6 expone las conclusiones derivadas del estudio.

2 DIAGNÓSTICO Y PLANTEAMIENTO DEL PROBLEMA

2.1. PLANTEAMIENTO DEL PROBLEMA

Los sistemas empresariales de reportes de ventas procesan continuamente solicitudes de múltiples usuarios con distintos roles —administradores, gerentes y vendedores— para generar informes de diversa complejidad: ventas diarias, mensuales, por producto, por sucursal y por cliente. Estas operaciones involucran una cadena de servicios que puede describirse de la siguiente manera:

Usuario → Report Service → Sales API → Base de Datos

Cada capa registra métricas de rendimiento propias, incluyendo tiempos de respuesta, cantidad de registros procesados, uso de CPU y memoria, códigos de estado HTTP, conteo de errores y número de reintentos. Bajo condiciones normales de operación, estas métricas se mantienen dentro de rangos esperados que varían según el tipo de reporte solicitado.

El problema central radica en la **detección oportuna de comportamientos anómalos** dentro de este flujo de datos. Cuatro categorías de anomalías afectan con mayor frecuencia a este tipo de sistemas:

1. **Consultas lentas a la base de datos** (*slow_database_query*): el tiempo de consulta se multiplica entre 8 y 20 veces respecto al valor esperado, con un aumento proporcional en las filas escaneadas. Esta anomalía suele indicar ausencia de índices adecuados, bloqueos de tabla o degradación del motor de base de datos.
2. **Exportaciones de reportes de gran tamaño** (*large_report_export*): la cantidad de registros retornados y el tamaño del reporte se incrementan entre 10 y 50 veces, provocando una presión excesiva sobre la memoria del servicio. Este patrón puede indicar filtros mal configurados o solicitudes maliciosas de extracción masiva de datos.
3. **Alta tasa de errores** (*high_error_rate*): los códigos de estado HTTP cambian a valores 5xx (500, 502, 503, 504), con múltiples reintentos por solicitud. Señala inestabilidad en el servicio o en la conectividad entre capas.
4. **Alto uso de recursos** (*high_resource_usage*): el uso de CPU supera el 85 % y el consumo de memoria se multiplica entre 3 y 7 veces, degradando la disponibilidad del servicio para otros usuarios.

El monitoreo manual de estos patrones resulta inviable a escala: un sistema con 120 usuarios activos y 10 000 solicitudes mensuales genera 30 000 entradas de log distribuidas en tres servicios. La revisión humana no puede operar en tiempo real, lo que genera una brecha entre la ocurrencia de una anomalía y su detección. La aplicación de algoritmos de minería de datos ofrece una solución escalable para automatizar este proceso de

vigilancia (Zhang et al., 2019).

3 OBJETIVO DE LA INVESTIGACIÓN

3.1. OBJETIVO GENERAL

Aplicar y comparar algoritmos de detección de anomalías no supervisados (*Isolation Forest*, *Local Outlier Factor* y *One-Class SVM*) sobre los registros de logs de un sistema de reportes de ventas distribuido, con el fin de identificar automáticamente comportamientos anómalos en el flujo Usuario → Report Service → Sales API → Base de Datos.

3.2. OBJETIVOS ESPECÍFICOS

1. Realizar un análisis exploratorio de los datos de logs del sistema de reportes de ventas, identificando la distribución de las variables numéricas, la prevalencia de cada tipo de anomalía y los patrones temporales de ocurrencia.
2. Preprocesar el conjunto de datos mediante codificación de variables categóricas y normalización de características numéricas para su uso como entrada en modelos de aprendizaje automático.
3. Implementar y entrenar tres modelos de detección de anomalías no supervisados: *Isolation Forest*, *Local Outlier Factor* con novedad y *One-Class SVM*, entrenados exclusivamente con registros normales.
4. Evaluar y comparar el desempeño de los tres modelos utilizando las métricas de precisión, *recall*, F1-Score y AUC-ROC, contrastando las predicciones contra las

etiquetas de verdad absoluta disponibles en el conjunto de datos.

5. Determinar las variables con mayor poder discriminativo para la detección de anomalías mediante análisis de importancia de características sobre el modelo de *Isolation Forest*.

4 ANÁLISIS DE LA INFORMACIÓN

4.1. FUENTE DE DATOS

El conjunto de datos utilizado en este estudio fue generado sintéticamente mediante un script de Python que simula el flujo de logs real de un sistema de reportes de ventas empresarial. Esta aproximación permite contar con etiquetas de verdad absoluta (*ground truth*) para la evaluación rigurosa de los modelos, siguiendo la metodología de Du et al. (2017) para la generación controlada de anomalías en logs.

El conjunto de datos contiene **30 000 registros**, distribuidos uniformemente entre tres servicios:

- **Report Service:** 10 000 registros (punto de entrada del usuario).
- **Sales API:** 10 000 registros (capa intermedia de lógica de negocio).
- **Base de Datos:** 10 000 registros (capa de persistencia).

Los registros corresponden a **10 000 trazas únicas** (*trace_id*), cada una representando una solicitud completa de reporte desde el usuario hasta la base de datos, durante el

período del 1 al 31 de enero de 2026. El sistema cuenta con 120 usuarios únicos distribuidos en 20 tenants, con roles de administrador, gerente y vendedor.

Cada registro incluye las siguientes variables:

Variable	Descripción
timestamp	Marca temporal del evento (ISO 8601)
trace_id	Identificador único de la traza de solicitud
service	Servicio que generó el log
user_hash	Identificador anonimizado del usuario
tenant_hash	Identificador anonimizado del tenant
user_role	Rol del usuario (admin, manager, seller)
report_type	Tipo de reporte solicitado
status_code	Código de estado HTTP de la respuesta
response_time_ms	Tiempo de respuesta total del servicio (ms)
db_query_time_ms	Tiempo de consulta a la base de datos (ms)
api_response_time_ms	Tiempo de respuesta de la Sales API (ms)
records_returned	Número de registros retornados
rows_scanned	Número de filas escaneadas en la DB
report_size_kb	Tamaño del reporte generado (KB)
cpu_usage_percent	Uso de CPU del servicio (%)
memory_usage_mb	Uso de memoria RAM (MB)
retry_count	Número de reintentos de la solicitud
error_count	Número de errores registrados
is_anomaly	Etiqueta binaria (0=normal, 1=anomalía)
anomaly_type	Tipo específico de anomalía o "normal"

Tabla 4.1: Variables del conjunto de datos de logs.

La distribución de anomalías en el conjunto de datos es la siguiente:

Tipo de anomalía	Cantidad	Porcentaje
Normal	28 635	95,45 %
Consulta lenta a DB (<i>slow_database_query</i>)	399	1,33 %
Alto uso de recursos (<i>high_resource_usage</i>)	339	1,13 %
Exportación grande (<i>large_report_export</i>)	315	1,05 %
Alta tasa de errores (<i>high_error_rate</i>)	312	1,04 %
Total	30 000	100 %

Tabla 4.2: Distribución de tipos de anomalía en el conjunto de datos.

4.2. PREPARACIÓN DE LOS DATOS

4.2.1. Exploración y Análisis Descriptivo

El análisis exploratorio inicial se centró en el servicio *report-service*, por ser el punto de entrada de las solicitudes y el que acumula los efectos de anomalías en las capas inferiores. Las estadísticas descriptivas de las principales variables numéricas son las siguientes:

Variable	Media	Desv. Est.	Mín.	Med.	Máx.
T. respuesta (ms)	3 995,77	5 021,09	933	3 084	96 135
T. consulta DB (ms)	1 429,42	2 236,65	128	1 074	60 700
Registros retornados	5 820,12	15 222,27	100	3 990	424 502
CPU (%)	43,21	13,96	20,00	42,91	98,91
Memoria (MB)	782,54	537,64	250	738	8 330

Tabla 4.3: Estadísticas descriptivas del servicio *report-service*.

La alta desviación estándar en variables como el tiempo de respuesta (5 021 ms frente a una media de 3 995 ms) y los registros retornados (15 222 frente a una media de 5 820) evidencia la presencia de valores extremos introducidos por las anomalías. El valor máximo del tiempo de respuesta (96 135 ms) corresponde a anomalías de consulta lenta,

mientras que el máximo de registros retornados (424 502) corresponde a exportaciones de gran tamaño.

La Figura 1 muestra la distribución de los tipos de anomalía tanto de manera global como por servicio.

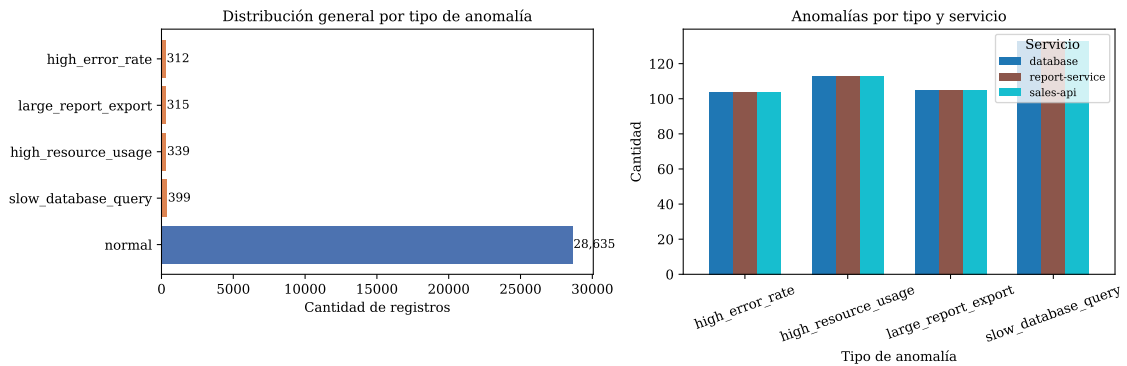


Figura 1: Distribución de tipos de anomalía: global (izquierda) y por servicio (derecha).

Se observa que la distribución de anomalías es uniforme entre los tres servicios, lo cual es esperable dado que cada anomalía se propaga a través de toda la cadena de servicios dentro de la misma traza. Las consultas lentas a la base de datos son ligeramente más frecuentes (399 instancias), seguidas por el alto uso de recursos (339), exportaciones grandes (315) y alta tasa de errores (312).

La Figura 2 presenta los diagramas de caja del tiempo de respuesta y del tiempo de consulta a la base de datos, segmentados por tipo de anomalía.

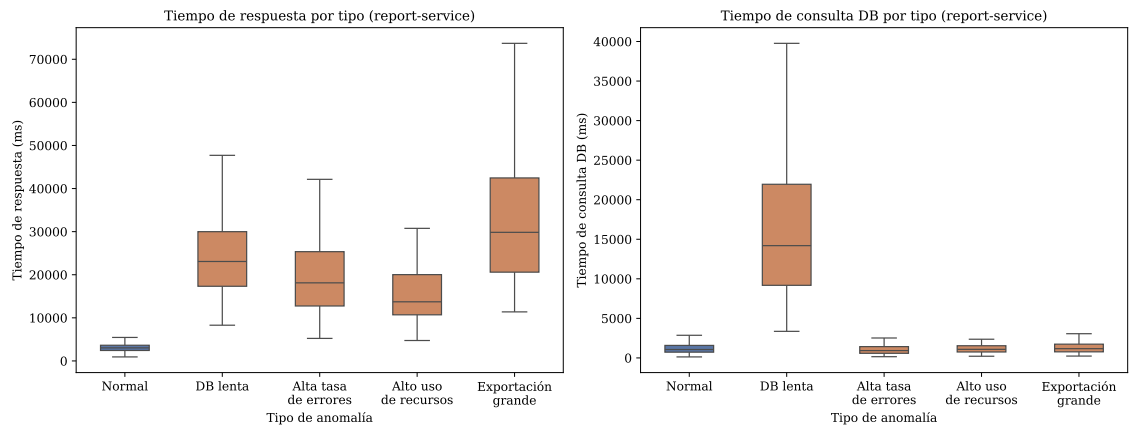


Figura 2: Distribución de tiempos de respuesta por tipo de anomalía en *report-service*.

La figura confirma que las anomalías de consulta lenta a la base de datos producen los tiempos de respuesta más elevados, seguidas por las exportaciones de gran tamaño. Las anomalías de alta tasa de errores y alto uso de recursos también muestran tiempos de respuesta significativamente superiores al caso normal, aunque con menor magnitud.

La Figura 3 presenta la matriz de correlación entre las variables numéricas del conjunto de datos.

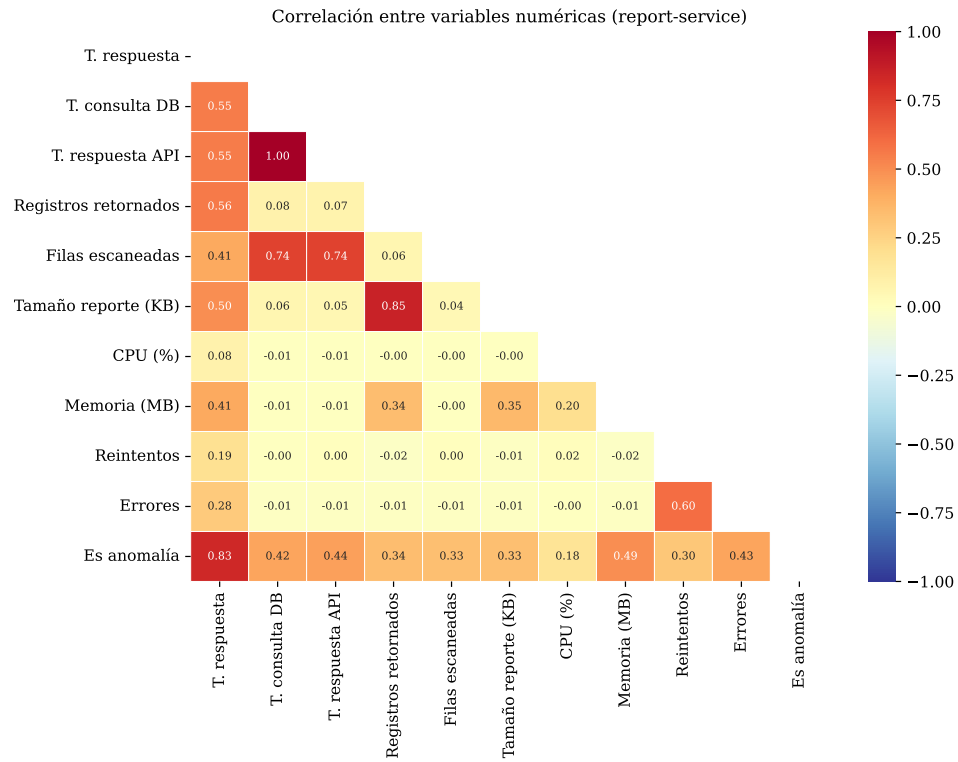


Figura 3: Matriz de correlación de variables numéricas (*report-service*).

Se observan altas correlaciones entre las variables de tiempo (tiempo de respuesta, tiempo de consulta a DB y tiempo de respuesta de API), lo que es coherente con la naturaleza en cascada del flujo de servicios. La variable *is_anomaly* presenta mayor correlación con las variables de tiempo y tamaño del reporte, lo cual sugiere que estas son las características más discriminativas para la detección.

La Figura 4 muestra la distribución temporal de registros y la tasa de anomalías por hora del día.

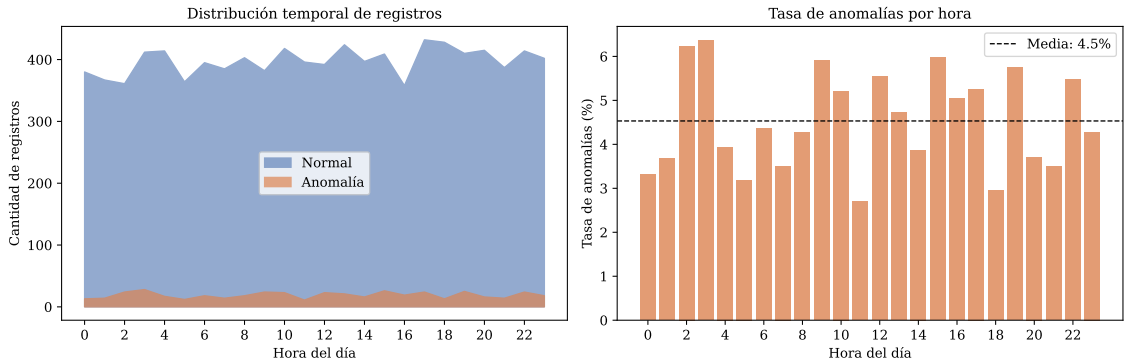


Figura 4: Patrones temporales: volumen de registros y tasa de anomalías por hora.

La tasa de anomalías se mantiene relativamente estable a lo largo del día, oscilando alrededor del 4,5 %, lo que indica que las anomalías no están concentradas en horarios específicos y responden a causas sistémicas más que a patrones temporales de carga.

4.2.2. Preprocesamiento y Selección de Características

El preprocesamiento incluyó los siguientes pasos:

1. **Codificación de variables categóricas:** las variables `service`, `report_type` y `user_role` fueron codificadas numéricamente mediante *Label Encoding*.
2. **Extracción de características temporales:** se extrajeron la hora del día (`hour`), el día del mes (`day`) y el día de la semana (`weekday`) a partir del campo `timestamp`.
3. **Tratamiento de valores ausentes:** los valores nulos resultantes del procesamiento de marcas temporales fueron reemplazados con cero.
4. **Normalización:** todas las características fueron estandarizadas mediante el método *StandardScaler* ($\mu = 0$, $\sigma = 1$) para garantizar que las diferencias de escala no afecten el comportamiento de los modelos.

El vector de características final contiene 14 variables:

```

response_time_ms, db_query_time_ms, api_response_time_ms,
records_returned, rows_scanned, report_size_kb, cpu_usage_percent,
memory_usage_mb, retry_count, error_count, service_enc, report_type_enc,
user_role_enc, hour

```

4.3. PROCESO DE MODELADO

Se adoptó un enfoque de detección de anomalías **no supervisado**, coherente con el contexto real donde las etiquetas de anomalía no están disponibles durante el entrenamiento (Chandola et al., 2009). Los modelos fueron entrenados exclusivamente con el 70 % de los registros normales (20 044 muestras), aprendiendo la distribución del comportamiento esperado del sistema. La evaluación se realizó sobre el conjunto completo de 30 000 registros, contrastando las predicciones contra las etiquetas de verdad absoluta disponibles.

4.3.1. Isolation Forest

El algoritmo *Isolation Forest* (Liu et al., 2008, 2012) se basa en el principio de que las anomalías son instancias atípicas que resultan más fáciles de aislar que las instancias normales. El algoritmo construye un conjunto de árboles de decisión (*isolation trees*), particionando aleatoriamente el espacio de características mediante la selección de un atributo y un punto de corte aleatorios. El número de particiones necesarias para aislar una instancia es su **puntuación de anomalía**: las anomalías requieren menos particiones y, por tanto, tienen caminos más cortos en el árbol.

La puntuación de anomalía normalizada se define como:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \quad (4.1)$$

donde $h(x)$ es la longitud del camino de x en el árbol, $c(n)$ es el valor esperado de $h(x)$ para un conjunto de datos de tamaño n , y $E[\cdot]$ es el valor esperado sobre todos los árboles del conjunto.

Los hiperparámetros utilizados fueron: `n_estimators=200`, `contamination=0.05`, `random_state=42`.

4.3.2. Local Outlier Factor

El algoritmo *Local Outlier Factor* (LOF) (Breunig et al., 2000) mide el grado en que una instancia es un valor atípico comparando la densidad local de sus vecinos más cercanos con la densidad local de sus propios vecinos. Una instancia con una densidad local considerablemente menor que la de sus vecinos tiene un LOF elevado y se considera una anomalía.

La densidad local de alcanzabilidad de una instancia p se define como:

$$lrd_k(p) = \left(\frac{\sum_{o \in N_k(p)} \text{reach-dist}_k(p, o)}{|N_k(p)|} \right)^{-1} \quad (4.2)$$

donde $N_k(p)$ es el conjunto de k vecinos más cercanos y $\text{reach-dist}_k(p, o) = \max\{k\text{-dist}(o), d(p, o)\}$.

El LOF de una instancia p se calcula como:

$$\text{LOF}_k(p) = \frac{\sum_{o \in N_k(p)} \frac{lrd_k(o)}{lrd_k(p)}}{|N_k(p)|} \quad (4.3)$$

Se utilizó la variante de **detección de novedad** (`novelty=True`), que permite aplicar el modelo entrenado sobre nuevas instancias. Los hiperparámetros fueron: `n_neighbors=30`, `contamination=0.05`.

4.3.3. One-Class SVM

El algoritmo *One-Class SVM* (Schölkopf et al., 2001) aprende una función de decisión que delimita la frontera de la distribución de datos normales en el espacio de características transformado por un kernel. El modelo encuentra el hiperplano de máximo margen que separa los datos de entrenamiento del origen, de modo que las instancias por fuera de esta región se clasifican como anomalías.

El problema de optimización se formula como:

$$\min_{w, \xi, \rho} \frac{1}{2} \|w\|^2 + \frac{1}{vn} \sum_{i=1}^n \xi_i - \rho \quad (4.4)$$

sujeto a $\langle w, \phi(x_i) \rangle \geq \rho - \xi_i$ y $\xi_i \geq 0$, donde $v \in (0, 1]$ controla la proporción máxima de anomalías y $\phi(\cdot)$ es la función de mapeado del kernel RBF.

Los hiperparámetros utilizados fueron: kernel=rbf, nu=0.05, gamma=scale.

4.4. EVALUACIÓN

Para evaluar el desempeño de los modelos se utilizaron cuatro métricas estándar de clasificación binaria (Fawcett, 2006; Tan et al., 2018):

- **Precisión:** fracción de instancias predichas como anomalías que efectivamente lo son.

$$\text{Precisión} = \frac{TP}{TP + FP} \quad (4.5)$$

- **Recall** (sensibilidad): fracción de anomalías reales que fueron correctamente detectadas.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4.6)$$

- **F1-Score:** media armónica de precisión y recall, equilibrando ambas métricas.

$$F1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} \quad (4.7)$$

- **AUC-ROC:** área bajo la curva ROC, que mide la capacidad discriminativa del modelo independientemente del umbral de clasificación.

Para el cálculo de la curva ROC, se utilizaron las puntuaciones de anomalía continuas de cada modelo, normalizadas al rango $[0, 1]$, siendo los valores más altos indicativos de mayor probabilidad de anomalía.

5 RESULTADOS

5.1. COMPARACIÓN DE MODELOS

La Tabla 5.1 presenta el resumen comparativo de las métricas de desempeño para los tres modelos evaluados.

Modelo	Precisión	Recall	F1-Score	AUC-ROC
Isolation Forest	0,3177	0,4960	0,3873	0,8626
Local Outlier Factor	0,5052	0,9985	0,6709	0,9995
One-Class SVM	0,4906	0,9919	0,6565	0,9974

Tabla 5.1: Comparación de métricas de desempeño por modelo.

El **Local Outlier Factor** obtiene el mejor desempeño en todas las métricas evaluadas, alcanzando un AUC-ROC de 0,9995 y un F1-Score de 0,6709. Este resultado es destacable considerando que los modelos fueron entrenados sin acceso a etiquetas de anomalía.

La Figura 5 presenta la comparación visual de las cuatro métricas para los tres modelos.

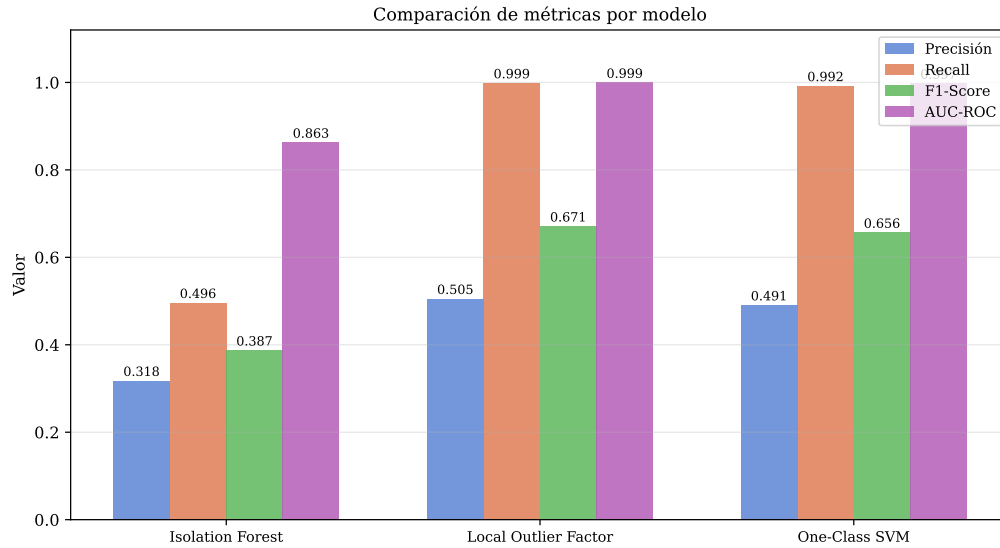


Figura 5: Comparación de métricas de desempeño por modelo.

5.2. CURVAS ROC

La Figura 6 muestra las curvas ROC de los tres modelos. Tanto el *Local Outlier Factor* (AUC = 0,9995) como el *One-Class SVM* (AUC = 0,9974) alcanzan una capacidad discriminativa casi perfecta, lo que indica que sus puntuaciones de anomalía separan efectivamente los registros normales de los anómalos. El *Isolation Forest* presenta un AUC inferior (0,8626), aunque aún superior al clasificador aleatorio.

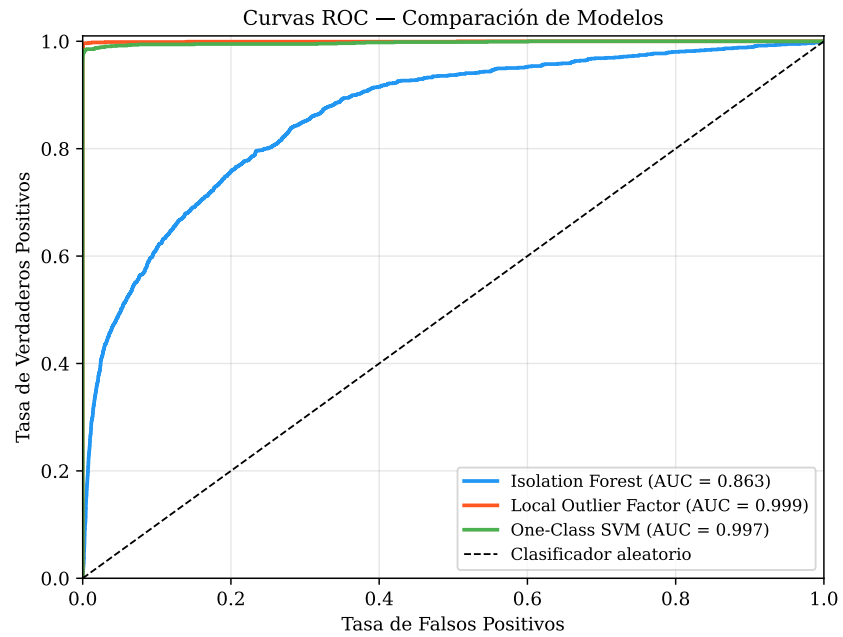


Figura 6: Curvas ROC para los tres modelos evaluados.

5.3. MATRICES DE CONFUSIÓN

La Figura 7 presenta las matrices de confusión para los tres modelos, evaluados sobre el conjunto completo de 30 000 registros.

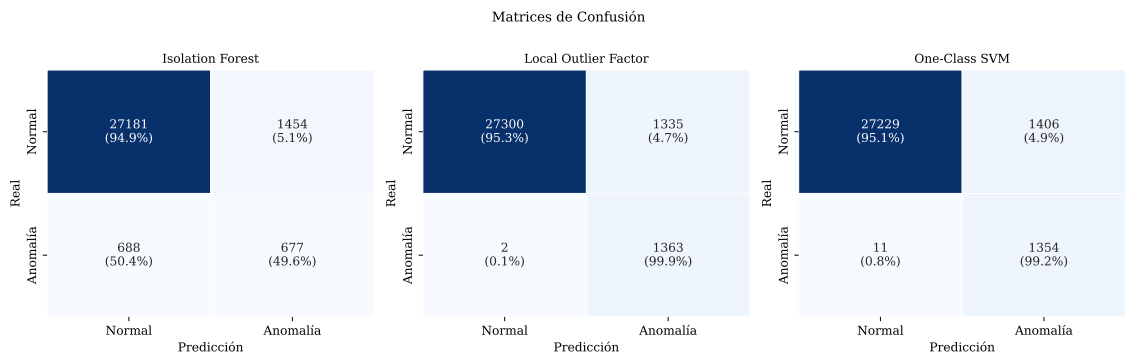


Figura 7: Matrices de confusión para Isolation Forest, LOF y One-Class SVM.

El *Local Outlier Factor* detecta prácticamente la totalidad de las anomalías reales (re-

call = 0,9985), a costa de un número moderado de falsos positivos. El *One-Class SVM* exhibe un comportamiento similar. El *Isolation Forest*, en cambio, produce menos falsos positivos pero pierde un mayor número de anomalías reales (recall = 0,4960).

5.4. IMPORTANCIA DE VARIABLES

La Figura 8 muestra la importancia de las 14 características según el análisis de permutación sobre el *Isolation Forest*.

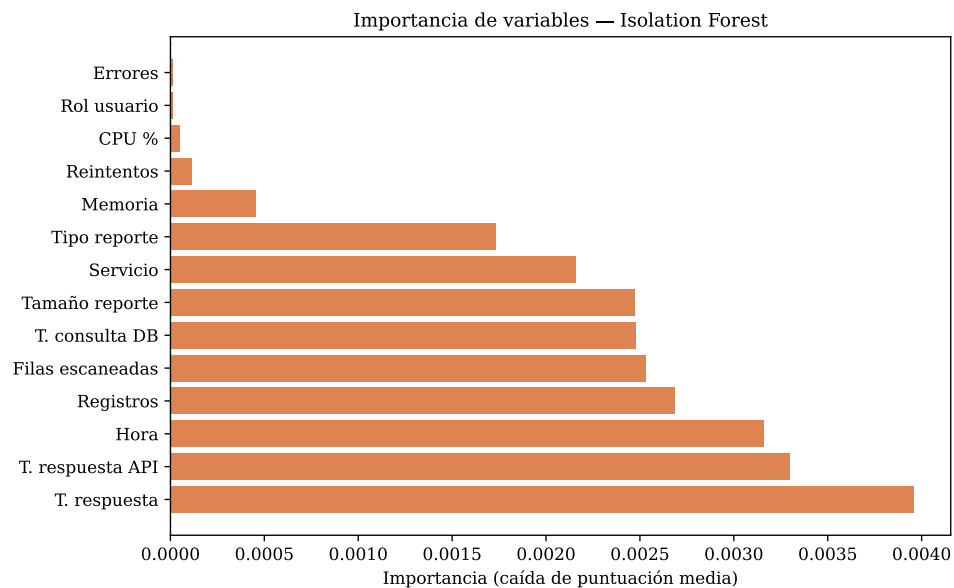


Figura 8: Importancia de variables por permutación en Isolation Forest.

Las cinco variables más importantes para la detección de anomalías son:

1. **Tiempo de respuesta total** (response_time_ms): importancia = 0,003957. Es la variable con mayor poder discriminativo, dado que todas las anomalías producen incrementos en el tiempo de respuesta del servicio de reporte.
2. **Tiempo de respuesta de la API** (api_response_time_ms): importancia = 0,003299. Correlacionado con el anterior, refleja la propagación del retardo desde la base de

datos hacia la capa API.

3. **Hora del día** (hour): importancia = 0,003161. Su relevancia sugiere variaciones en el comportamiento normal del sistema según el momento del día.
4. **Registros retornados** (records_returned): importancia = 0,002688. Clave para detectar exportaciones de gran tamaño.
5. **Filas escaneadas** (rows_scanned): importancia = 0,002530. Indicador directo de consultas ineficientes a la base de datos.

5.5. DETECCIÓN POR SERVICIO

La Figura 9 muestra el número de anomalías reales, predichas y verdaderos positivos por servicio para el mejor modelo (*Local Outlier Factor*).

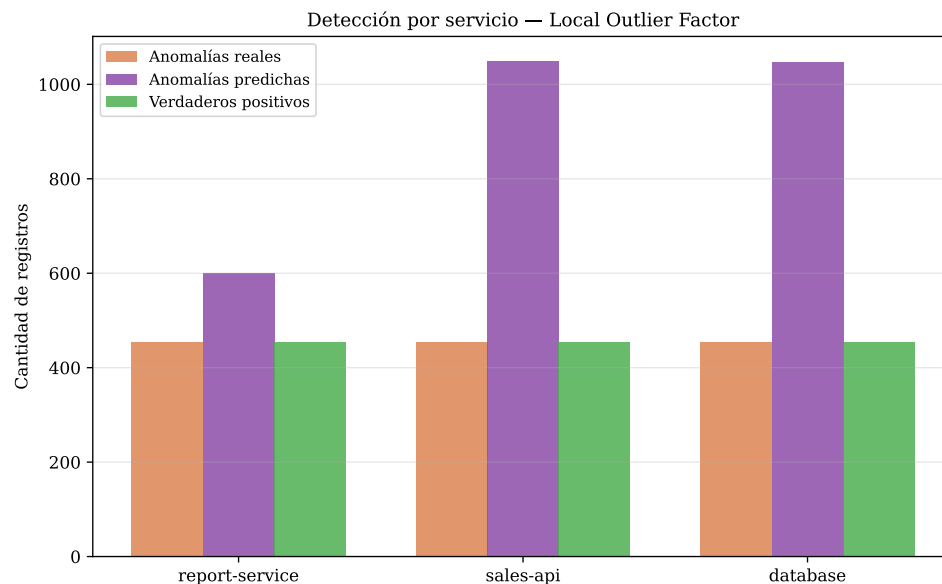


Figura 9: Detección de anomalías por servicio — Local Outlier Factor.

El modelo mantiene un desempeño consistente en los tres servicios, detectando la gran mayoría de las anomalías en cada capa del flujo de servicios. El número de verdaderos

positivos es aproximadamente igual en los tres servicios, lo que confirma que el modelo generaliza bien independientemente de las características específicas de cada capa.

6 CONCLUSIONES

Este trabajo aplicó tres algoritmos de detección de anomalías no supervisados sobre un conjunto de 30 000 registros de logs de un sistema distribuido de reportes de ventas. Las principales conclusiones son:

1. El **Local Outlier Factor** demostró el mejor desempeño general, alcanzando un AUC-ROC de 0,9995 y un F1-Score de 0,6709. Su capacidad para medir la densidad local de cada instancia en relación con sus vecinos lo hace especialmente eficaz para detectar anomalías en datos con distribuciones no uniformes como los logs de sistemas de reportes.
2. El **One-Class SVM** obtuvo resultados comparables al LOF (AUC-ROC = 0,9974, F1 = 0,6565), siendo una alternativa robusta particularmente adecuada cuando se requiere una frontera de decisión matemáticamente bien definida.
3. El **Isolation Forest** presentó el desempeño más bajo en este conjunto de datos (AUC-ROC = 0,8626, F1 = 0,3873). Sin embargo, este resultado refleja la mayor sensibilidad del modelo al parámetro de contaminación y a la naturaleza del conjunto de datos. Su ventaja radica en la escalabilidad: al no requerir cálculos de distancia, es significativamente más rápido que LOF para grandes volúmenes de datos.
4. El análisis de importancia de variables confirmó que el **tiempo de respuesta total**

y el **tiempo de consulta a la base de datos** son los indicadores más discriminativos, seguidos por los registros retornados y las filas escaneadas. Esto sugiere que el monitoreo de tiempos de respuesta debe ser la primera línea de alerta en este tipo de sistemas.

5. Los cuatro tipos de anomalía fueron detectados con diferentes grados de eficacia. Las anomalías de *slow_database_query* y *large_report_export* producen las señales más claras en las variables de tiempo y volumen de datos, siendo las más fáciles de detectar. Las de *high_error_rate* y *high_resource_usage* presentan perfiles más sutiles y requieren el uso combinado de múltiples características.
6. Desde una perspectiva práctica, se recomienda el uso de **Local Outlier Factor** o **One-Class SVM** para sistemas de detección en tiempo real con volúmenes moderados de datos (hasta decenas de miles de registros), y **Isolation Forest** cuando la escalabilidad sea el factor determinante, posiblemente ajustando el parámetro de contaminación mediante validación cruzada.

7 BIBLIOGRAFÍA

Bibliografía

Breunig, M. M., Kriegel, H.-P., Ng, R. T., and Sander, J. (2000). LOF: Identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD Internatio-*

- nal Conference on Management of Data*, pages 93–104. ACM.
- Chandola, V., Banerjee, A., and Kumar, V. (2009). Anomaly detection: A survey. In *ACM Computing Surveys*, volume 41, pages 1–58. ACM.
- Du, M., Li, F., Zheng, G., and Srikumar, V. (2017). Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1285–1298. ACM.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.
- He, P., Zhu, J., He, S., Li, J., and Lyu, M. R. (2016). An evaluation study on log parsing and its use in log mining. In *2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 654–661. IEEE.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422. IEEE.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2012). Isolation-based anomaly detection. *ACM Transactions on Knowledge Discovery from Data*, 6(1):1–39.
- Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., and Williamson, R. C. (2001). Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471.
- Tan, P.-N., Steinbach, M., Karpatne, A., and Kumar, V. (2018). *Introduction to Data Mining*. Pearson Education, 2nd edition.
- Xu, W., Huang, L., Fox, A., Patterson, D., and Jordan, M. I. (2009). Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, pages 117–132. ACM.

Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., et al. (2019). Robust log-based anomaly detection on unstable log data. *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 807–817.